
Université Internationale de Rabat

MIAGE — Semestre 8
Année universitaire 2025-2026

Module : Cloud Data Warehousing

TP4 — Fragmentation Horizontale Distribuée sur Oracle 19c Multitenant

Schéma OE sur architecture multi-PDB

Étudiant : Mohammed saij

Filière : Big Data & IA

Semestre : S8

Date : Avril 2026

1 Architecture du TP

Ce TP met en œuvre une fragmentation horizontale de la table `ORDERS` du schéma `OE`, distribuée sur trois PDB distinctes d'une instance Oracle 19c Multitenant. La table source réside dans `PDB1` ; les fragments sont réparés selon le critère du montant total des commandes (`ORDER_TOTAL`).

tableheader

	Source OLTP (schéma OE)	OE, OE_RO
PDB1		
	Fragment A — Commandes < 1000	FRAG_A
PDB2		
	Fragment B — Commandes ≥ 1000	FRAG_B
PDB3		

Table 1: Répartition des rôles entre les PDB de l'instance Multitenant

2 Réalisation

2.1 Préparation de la source (PDB1)

Dans `PDB1`, un utilisateur dédié en lecture seule `OE_RO` a été créé avec un quota de 50 Mo sur le tablespace `USERS`. Les privilèges `SELECT` ont été accordés sur les tables `CUSTOMERS`, `ORDERS` et `ORDER_ITEMS` du schéma `OE`, afin de permettre aux fragments distants d'accéder aux données source via DB Link sans pouvoir les modifier.

2. Réalisation

Partie 1 - Préparation de la source (PDB1)

Dans `PDB1`, un utilisateur de lecture `OE_RO` a été créé avec un quota de 50M sur le tablespace `USERS`. Les droits de lecture ont été accordés sur les tables `CUSTOMERS`, `ORDERS` et `ORDER_ITEMS` du schéma `OE`, afin de permettre aux fragments distants d'accéder aux données source via DB Link sans pouvoir les modifier.

```
SQL> CREATE USER OE_RO IDENTIFIED BY Welcome1 DEFAULT TABLESPACE USERS QUOTA 50M ON USERS;
User created.

SQL> GRANT CREATE SESSION TO OE_RO;
Grant succeeded.

SQL> GRANT SELECT ON OE.CUSTOMERS TO OE_RO;
```

Figure 1: Création de l'utilisateur `OE_RO` et attribution des droits de lecture sur le schéma `OE` dans `PDB1`

2.2 Fragmentation dans PDB2 — Fragment A

Dans PDB2, l'utilisateur FRAG_A a été créé et doté des privilèges nécessaires (CREATE SESSION, CREATE TABLE, CREATE DATABASE LINK, CREATE VIEW). Un DB Link LNK_PDB1_OE a été établi vers PDB1, permettant à FRAG_A d'accéder au schéma OE via OE_RO. Le test de connectivité confirme l'accès aux 5 commandes de la table source.

La table FRAG_ORDERS_LOW a ensuite été créée par sélection des commandes dont le montant est inférieur à 1000 (ORDER_TOTAL < 1000).

```
SQL> GRANT CREATE SESSION, CREATE TABLE, CREATE DATABASE LINK, CREATE VIEW TO FRAG_A;
Grant succeeded.

SQL> CONNECT FRAG_A/welcome1@localhost:1521/pdb2
Connected.
SQL> CREATE DATABASE LINK LNK_PDB1_OE CONNECT TO OE_RO IDENTIFIED BY welcome1 USING 'localhost:1521/pdb1';
Database link created.

SQL> SELECT COUNT(*) FROM OE.ORDERS@LNK_PDB1_OE;

COUNT(*)
-----
5
```

Figure 2 - Création du DB Link LNK_PDB1_OE dans PDB2 et vérification de la connexion (COUNT = 5)

Figure 2: Création du DB Link LNK_PDB1_OE dans PDB2 et vérification de la connexion (COUNT = 5)

```
Table created.

SQL> SELECT COUNT(*) FROM FRAG_ORDERS_LOW;

COUNT(*)
-----
2
```

Figure 3 - Création de FRAG_ORDERS_LOW dans PDB2 : 2 commandes de faible montant

Partie 3 - Fragmentation dans PDB3 (Fragment B)

De manière symétrique, l'utilisateur FRAG_B a été créé dans PDB3 avec un DB Link identique vers PDB1. La table FRAG_ORDERS_HIGH contient les commandes de montant supérieur ou égal à 1000.

Figure 3: Création de FRAG_ORDERS_LOW dans PDB2 : 2 commandes de faible montant

2.3 Fragmentation dans PDB3 — Fragment B

De manière symétrique, l'utilisateur FRAG_B a été créé dans PDB3 avec un DB Link identique vers PDB1. La table FRAG_ORDERS_HIGH contient les commandes dont le montant est supérieur ou égal à 1000 (ORDER_TOTAL >= 1000).

```
-----
3
```

Figure 4 - Création de FRAG_ORDERS_HIGH dans PDB3 : 3 commandes de montant élevé

Parties 4 et 5 - Interconnexion et reconstitution globale

Des DB Links croisés ont été établis entre PDB2 et PDB3 (LNK_PDB3 depuis PDB2, LNK_PDB2 depuis PDB3), permettant à chaque fragment d'accéder à l'autre. Sur cette base, une vue V_GLOBAL_ORDERS a été créée dans chacune des deux PDB, reconstituant l'ensemble des commandes par union des deux fragments.

Figure 4: Création de FRAG_ORDERS_HIGH dans PDB3 : 3 commandes de montant élevé

2.4 Interconnexion et reconstructions globale

Des DB Links croisés ont été établis entre PDB2 et PDB3 (LNK_PDB3 depuis PDB2, LNK_PDB2 depuis PDB3), permettant à chaque fragment d'accéder à l'autre. Sur cette base, une vue V_GLOBAL_ORDERS a été créée dans chacune des deux PDB, reconstituant l'intégralité des commandes par UNION ALL des deux fragments.

```
SQL> CREATE VIEW V_GLOBAL_ORDERS AS SELECT * FROM FRAG_ORDERS_LOW UNION ALL SELECT * FROM FRAG_ORDERS_HIGH@LNK_PDB3;
View created.
SQL> SELECT COUNT(*) FROM V_GLOBAL_ORDERS;
COUNT(*)
-----
5
```

Figure 5: Création de V_GLOBAL_ORDERS dans PDB2 par UNION ALL des deux fragments (COUNT = 5)

Figure 6 - Création de V_GLOBAL_ORDERS dans PDB3 par UNION ALL des deux fragments (COUNT = 5)

```
SQL> SELECT COUNT(*) FROM V_GLOBAL_ORDERS;
COUNT(*)
-----
5
SQL> |
```

Figure 6 - Vue globale dans PDB3 confirmant également la reconstitution complète (COUNT = 5)

Partie 6 - Vérification de la cohérence

La vérification finale compare le volume de la table source dans PDB1 avec la somme des

Figure 6: Vue globale dans PDB3 confirmant également la reconstitution complète (COUNT = 5)

2.5 Vérification de la cohérence

La vérification finale compare le volume de la table source dans PDB1 avec la somme des fragments et le résultat de la vue globale. Les trois valeurs sont identiques, ce qui confirme la cohérence de la fragmentation.

cohérence de la fragmentation.

```
SQL> SELECT COUNT(*) FROM OE.ORDERS;
COUNT(*)
-----
5
```

Figure 7 - COUNT(*) sur OE.ORDERS dans PDB1 : 5 enregistrements, cohérent avec 2 (PDB2) + 3 (PDB3)

Vérification	Résultat
--------------	----------

Figure 7: COUNT(*) sur OE.ORDERS dans PDB1 : 5 enregistrements, cohérent avec 2 (PDB2) + 3 (PDB3)

tableheader

OE.ORDERS dans PDB1 (source)	5
FRAG_ORDERS_LOW dans PDB2	2
(ORDER_TOTAL < 1000)	
FRAG_ORDERS_HIGH dans PDB3 (ORDER_TOTAL	3

≥ 1000)

uirblue!10 V_GLOBAL_ORDERS (vue reconstruite) 5

Table 2: Tableau récapitulatif de la vérification de cohérence

3 Réponses aux questions théoriques

3.0 Q1 — Pourquoi parle-t-on de fragmentation horizontale ?

La fragmentation est dite horizontale car la distribution porte sur les *lignes* (tuples) de la table, et non sur ses colonnes. Les fragments PDB2 et PDB3 possèdent exactement le même schéma relationnel que la table source OE.ORDERS : mêmes attributs, mêmes types. Ce qui diffère, c'est le sous-ensemble de tuples stocké dans chaque fragment :

- PDB2 — commandes de faible montant : $\text{ORDER_TOTAL} < 1000$
- PDB3 — commandes de montant élevé : $\text{ORDER_TOTAL} \geq 1000$

Les deux fragments sont disjoints et leur réunion reconstruit exactement la table d'origine, sans perte ni duplication de données — propriétés fondamentales d'une fragmentation correcte.

3.0 Q2 — Quelle différence entre fragmentation logique et physique ?

Fragmentation logique — désigne la partition *conceptuelle* des données selon un critère métier (ici le montant de la commande). Elle définit les règles de répartition indépendamment de tout support de stockage.

Fragmentation physique — désigne la *matérialisation concrète* de ces fragments dans des espaces de stockage distincts. Dans ce TP, chaque fragment est une table matérielle stockée dans une PDB distincte avec ses propres fichiers de données sur le système de fichiers de la VM.

Dans ce TP, la fragmentation est à la fois logique (critère ORDER_TOTAL) et physique (tables réelles dans des PDB séparées).

3.0 Q3 — Avantages et limites de cette architecture

Performances Chaque nœud traite un volume réduit de données, ce qui accélère les requêtes locales et réduit la charge globale.

L'ajout de nouveaux fragments permet de distribuer la charge sans restructurer le schéma existant.

Isolation des pannes

Une panne sur PDB2 n'affecte pas PDB3 ; les données restent par conséquent accessibles.

Isolation des pannes

La vue V_GLOBAL_ORDERS masque la distribution aux applications clientes.

Transparence

Les requêtes croisant les deux fragments génèrent des transferts via DB Link, pouvant dégrader les performances.

8 Transferts réseau

L'absence de mécanisme transactionnel distribué rend difficile la gestion des mises à jour simultanées sur les deux PDB.

5 Cohérence

manuelle

Table 3: Avantages et limites de l'architecture de fragmentation horizontale